

Computational Imaging

DeBlur & Denoise di Immagini di Citologia Cervicale LBC

Copione per l'Esposizione Orale

Gruppo R – Francesco Castaldi, Paolo Fusco

A.A. 2025/2026

Questo documento accompagna `slides/presentazione.tex` e contiene il copione dettagliato per ogni slide, con collegamenti alla teoria del corso.

Da stampare e portare all'orale come traccia di esposizione.

Indice

1	Slide 1 â€” Title Page	3
2	Slide 2 â€” Problema Inverso	3
3	Slide 3 â€” Metodi Scelti	4
4	Slide 4 â€” Mendeley LBC Cervical Cancer	4
5	Slide 5 â€” Preprocessing e Split	5
6	Slide 6 â€” Degradazione	6
7	Slide 7 â€” Total Variation: Teoria	6
8	Slide 8 â€” Total Variation: Algoritmo	7
9	Slide 9 â€” Total Variation: Risultati	8
10	Slide 10 â€” UNet: Architettura	8
11	Slide 11 â€” UNet: Training	9
12	Slide 12 â€” UNet: Risultati	10
13	Slide 13 â€” DiffPIR: Panoramica	11
14	Slide 14 â€” DDPM e LightUNet	11
15	Slide 15 â€” Algoritmo DiffPIR	12
16	Slide 16 â€” DiffPIR: Risultati	13
17	Slide 17 â€” Implementazione	14
18	Slide 18 â€” Confronto PSNR/SSIM	14
19	Slide 19 â€” Grafico Comparativo	15
20	Slide 20 â€” Griglia Completa	16
21	Slide 21 â€” Confronto Visivo $\sigma_n = 0.01$	16
22	Slide 22 â€” Confronto Visivo $\sigma_n = 0.05$	16
23	Slide 23 â€” Confronto Visivo $\sigma_n = 0.1$	17
24	Slide 24 â€” Confronto tra le Famiglie	17
25	Slide 25 â€” Regimi di Funzionamento	18
26	Slide 26 â€” Conclusioni	18
27	Slide 27 â€” Riferimenti	18

1 Slide 1 – Title Page

Slide: “Computational Imaging – Deblur e Denoise di Immagini di Citologia Cervicale LBC”

Collegamenti Teorici

Riferimenti: Problema Inverso (theory.md §1), Dataset (theory.md §2), Definizione del task (report.md §3.1).

Copione

“Buongiorno, siamo il Gruppo R, composto da Francesco Castaldi e Paolo Fusco. Presentiamo il nostro progetto di Computational Imaging: deblur e denoise di immagini di citologia cervicale provenienti dal dataset Mendeley LBC Cervical Cancer.

Il progetto confronta tre diverse famiglie metodologiche: un approccio variazionale (Total Variation), uno di deep learning end-to-end (UNet), e uno generativo (DiffPIR). L’obiettivo – duplice: risolvere il problema di restauro e, soprattutto, confrontare criticamente i tre approcci nelle stesse condizioni sperimentali.

Il codice sorgente – disponibile su GitHub all’indirizzo mostrato in slide.”

Note per l’esame

Potenziiali domande:

- “Perché avete scelto questi tre metodi e non Weighted TV?”
- “Qual – il contributo principale del progetto?”

Risposte suggerite:

- Il regolamento richiedeva un metodo per studente; essendo in 2, abbiamo scelto 3 metodi su 4, escludendo Weighted TV perché ibrido e meno distintivo.
- Il contributo principale – il confronto equo e sistematico tra tre famiglie, non la performance assoluta.

2 Slide 2 – Problema Inverso

Slide: “Problema Inverso”

Collegamenti Teorici

Riferimenti: theory.md §1 (intero), report.md §3.1.

Concetti chiave:

- Problema diretto vs. inverso: dato x calcolare y (diretto, ben posto) vs. dato y stimare x (inverso, mal posto).
- Modello di degradazione: $y = \mathcal{H}(x) + n$ con operatore di blur $\mathcal{H}$ e rumore AWGN n .
- Hadamard (1902): un problema – ben posto se soluzione esiste, – unica e dipende continuamente dai dati. Qui nessuna delle tre condizioni – garantita.
- Ill-posedness: soluzione non unica perché blur cancella informazioni ad alta frequenza. La regolarizzazione (TV, prior appresi) restringe lo spazio delle soluzioni.

Copione

“Partiamo dal problema inverso. Data un’immagine di citologia cervicale x , che rappresenta la ground truth sconosciuta, applichiamo un operatore di blur gaussiano $\mathcal{H}$, con sigma 2 e kernel 9 per 9, e aggiungiamo rumore AWGN a diversi livelli. Il risultato – l’osservazione y .

Il problema – stimare l’immagine originale x a partire dalla sola y .

Questo – un classico problema inverso mal posto secondo la definizione di Hadamard: esistono infinite immagini x compatibili con la stessa y , perché il blur ha cancellato le informazioni ad alta frequenza – i dettagli fini delle cellule – e il rumore aggiunge ambiguità. Non possiamo quindi limitarci a invertire l’operatore di blur perché amplificherebbero il rumore.

Servono regolarizzazioni o prior che vincolino la soluzione verso immagini realistiche. E qui si inseriscono i tre metodi: la TV usa un prior analitico di regolarizzazione delle variazioni locali; UNet

impara il prior dai dati in modo supervisionato; DiffPIR usa un prior generativo appreso tramite diffusion models.”

Note per l’esame

Potenziali domande:

- “Cosa significa esattamente ‘mal posto’?”
- “Perché il blur rende il problema mal posto?”
- “Qual è la differenza tra regolarizzazione e prior appreso?”

Questa slide è concettualmente la più importante: ogni domanda successiva ricollega questi concetti.

3 Slide 3 “Metodi Scelti

Slide: “Metodi Scelti”

Collegamenti Teorici

Riferimenti: report.md §1 (descrizione generale), src/methods/README.md, theory.md §3.

Classificazione metodologica:

- **Variazionale (TV):** minimizza un funzionale composto da data-fidelity + λ · regolarizzazione. Non richiede training.
- **Deep Learning (UNet):** apprende una mappatura end-to-end da coppie (degradata, GT). Richiede training supervisionato.
- **Generativo (DiffPIR):** usa un modello di diffusione come prior plug-and-play, alternando denoising e data-fidelity via FFT.

Copione

“Il progetto copre tre famiglie metodologiche.

La Total Variation è un metodo variazionale classico: minimizza una funzione obiettivo che bilancia fedeltà ai dati e regolarizzazione. È interpretabile, non richiede training, ma la qualità è limitata.

L’UNet è un metodo end-to-end di deep learning. Impara direttamente dai dati la mappatura da immagine degradata a immagine restaurata. L’inferenza è velocissima, ma richiede training supervisionato.

DiffPIR è un metodo generativo basato su modelli di diffusione. Integra un DDPM in un framework plug-and-play, alternando passi di denoising e di fedeltà ai dati risolta in frequenza. Produce buoni risultati ad alto rumore ma è computazionalmente più costoso.

Paolo ha implementato TV e UNet, Francesco DiffPIR.”

Note per l’esame

Domanda probabile: “Perché non avete fatto Weighted TV?” **Risposta:** Weighted TV è un metodo ibrido tra variazionale e deep learning. Per un gruppo di 2 studenti, 3 metodi su 4 sono già rappresentativi delle diverse famiglie.

4 Slide 4 “Mendeley LBC Cervical Cancer

Slide: “Mendeley LBC Cervical Cancer”

Collegamenti Teorici

Riferimenti: theory.md §2 (dataset), report.md §2 (dataset completo).

Dettaglio classi:

- NILM (Negative for Intraepithelial Malignancy): 63.6% “cellule normali
- HSIL (High-grade SIL): 16.9% “lesione di alto grado
- LSIL (Low-grade SIL): 11.7% “lesione di basso grado

- SCC (Squamous Cell Carcinoma): 7.7% “ carcinoma

Il dataset “ sbilanciato ma per il task di restauro questo non “ un problema, poich“ la variet“ morfologica “ rappresentata in tutte le classi.

Copione

“Il dataset utilizzato “ Mendeley LBC Cervical Cancer, composto da 962 immagini di citologia cervicale in fase liquida. Le immagini originali sono 2048 per 1536 pixel, che ridimensioniamo a 256 per 256 per ragioni computazionali.

Le immagini sono classificate in 4 categorie diagnostiche: NILM, che sono cellule normali e rappresentano la maggioranza; HSIL e LSIL, che sono lesioni intraepiteliali di alto e basso grado; e SCC, carcinoma squamoso.

La distribuzione “ sbilanciata “ NILM supera il 63% “ ma per il task di restauro delle immagini questo non costituisce un problema: non stiamo facendo classificazione, e la variet“ morfologica “ comunque presente in tutte le classi.

A destra vediamo un esempio di confronto tra ground truth, immagine degradata e restauro con i tre metodi.”

Note per l’esame

Domande possibili:

- “Perch“ 962 immagini e non 4000 come menzionato?”
- “Perch“ il resize a 256?”

Risposte: Il dataset Mendeley ha 962 immagini totali; abbiamo usato tutto il dataset. Il resize a 256 “ un compromesso tra costo computazionale e preservazione dei dettagli cellulari (nuclei, cromatina).

5 Slide 5 “ Preprocessing e Split

Slide: “Preprocessing e Split”

Collegamenti Teorici

Riferimenti: report.md “2.2, theory.md “6.

Pipeline:

1. Resize bilineare $2048 \rightarrow 256$: riduzione carico computazionale.
2. ToTensor $[0, 255] \rightarrow [0, 1]$: conversione in float per PyTorch.
3. Normalizzazione $(x - 0.5)/0.5 \rightarrow [-1, 1]$: range richiesto dal DDPM (coerente con MSE loss per immagini in questo range).
4. Split stratificato 70/15/15 con seed 42.

Copione

“La pipeline di preprocessing “ unica per tutti i metodi.

Primo passo: resize bilineare da 2048 per 1536 a 256 per 256. Perdiamo risoluzione ma il carico computazionale diventa gestibile, e i dettagli cellulari principali “ nuclei e cromatina “ rimangono visibili.

Secondo passo: convertiamo i pixel da interi 0–255 a float 0–1 per compatibilit“ con PyTorch.

Terzo passo: normalizziamo nell’intervallo $[-1, 1]$ dividendo per 0.5 e sottraendo 1. Questo range “ necessario per il modello DDPM e coerente con la loss L1 e MSE che lavorano meglio su segnali centrati.

Infine, dividiamo il dataset in training 70%, validation 15% e test 15%, con split stratificato per classe per mantenere le proporzioni e seed fisso 42 per riproducibilit“ .

Tutti i metodi usano lo stesso identico test set di 145 immagini.”

Note per l’esame

Domanda: “Perch“ normalizzate in $[-1, 1]$ e non $[0, 1]$?” **Risposta:** Il modello DDPM “ addestrato per generare dati con rumore gaussiano centrato; il range $[-1, 1]$ “ simmetrico e si sposa meglio con la predizione del rumore MSE. Inoltre, l’output con tanh dell’UNet produce naturalmente valori in $[-1, 1]$.

6 Slide 6 “Degradazione”

Slide: “Degradazione”

Collegamenti Teorici

Riferimenti: `theory.md` §3.1, `report.md` §3.2, `src/degradation/README.md`.

Parametri identici per tutti i metodi:

- Blur gaussiano: kernel 9×9 , $\sigma_{\text{blur}} = 2$. La scelta di $\sigma = 2$ produce uno sfocamento moderato che cancella dettagli fini (membrane cellulari, bordi nucleari) senza rendere l'immagine irriconoscibile.
- AWGN: $\sigma_n \in \{0.005, 0.01, 0.05, 0.1\}$. 4 livelli permettono di studiare la transizione tra regimi.

Copione

“Questa slide mostra la pipeline di degradazione, che $\hat{A}$ è identica per tutti i metodi. $\hat{A}$ è il punto fondamentale per garantire un confronto equo: le differenze nei risultati sono dovute solo ai metodi, non alla degradazione.

Applichiamo prima un blur gaussiano con sigma 2 e kernel 9 per 9. Abbiamo scelto sigma 2 perché produce una sfocatura visibile ma non estrema, cancellando dettagli come membrane cellulari e bordi dei nuclei senza rendere l'immagine irriconoscibile.

Poi aggiungiamo rumore AWGN a quattro livelli di deviazione standard: 0.005, 0.01, 0.05 e 0.1. Questi quattro livelli ci permettono di studiare come si comportano i metodi in regimi diversi: da rumore trascurabile a rumore significativo.

La pipeline $\hat{A}$ è implementata in un unico file `src/degradation/degradation.py`, usato da tutti i metodi. Il seed 42 garantisce riproducibilità.

A destra vediamo una striscia di immagini che mostra l'effetto dei diversi livelli di rumore su un crop.”

Note per l'esame

Domanda: “Perché proprio questi quattro livelli di rumore?” **Risposta:** I quattro livelli coprono regimi qualitativamente diversi: 0.005 $\hat{A}$ è rumore trascurabile (PSNR teoricamente alto), 0.01 $\hat{A}$ è rumore leggero, 0.05 $\hat{A}$ è rumore moderato, 0.1 $\hat{A}$ è rumore pesante. Questo permette di vedere la transizione completa del comportamento dei metodi.

7 Slide 7 “Total Variation: Teoria

Slide: “Total Variation “ Teoria”

Collegamenti Teorici

Riferimenti: `theory.md` §3.1, `report.md` §4.1.

Formulazione matematica:

$$\hat{x} = \arg \min_x \underbrace{\|H * x - y\|_2^2}_{\text{data fidelity}} + \lambda \cdot \underbrace{TV(x)}_{\text{regolarizzazione}}$$

Total Variation:

$$TV(x) = \sum_{i,j} |x_{i+1,j} - x_{i,j}| + |x_{i,j+1} - x_{i,j}|$$

Proprietà :

- La norma L1 del gradiente preserva i bordi (a differenza della regolarizzazione Tikhonov che usa norma L2 e sfuma i bordi).
- Effetto staircasing: a λ troppo alto, l'immagine sviluppa artefatti a gradino nelle regioni a gradiente dolce.
- Convexità : il problema $\hat{A}$ è convesso, quindi la soluzione $\hat{A}$ è globalmente ottima indipendentemente dall'inizializzazione.

Copione

“Cominciamo con il primo metodo: la Total Variation.

L'idea $\hat{x}$ semplice: cerchiamo l'immagine x che minimizza un funzionale composto da due termini. Il primo termine $\hat{x}$ la fedeltà ai dati: l'immagine x deve essere coerente con l'osservazione y dopo aver applicato il blur H . Il secondo termine $\hat{x}$ la regolarizzazione TV, che penalizza le variazioni locali.

La TV $\hat{x}$ la somma, su tutti i pixel, delle differenze in valore assoluto tra pixel adiacenti, sia in orizzontale che in verticale. $\hat{x}$ essenzialmente la norma L1 del gradiente dell'immagine. Usare la norma L1, invece della L2, $\hat{x}$ una scelta fondamentale: permette di preservare i bordi netti. Se usassimo L2 $\hat{x}$ che $\hat{x}$ la regolarizzazione di Tikhonov $\hat{x}$ i bordi verrebbero sfumati, perché la norma L2 penalizza molto di più le grandi differenze.

Il parametro λ controlla il bilanciamento. Con λ troppo basso, il rumore residuo $\hat{x}$ visibile. Con λ troppo alto, appare l'effetto staircasing: le regioni a gradiente dolce sviluppano artefatti a gradino.

Un vantaggio importante: il problema $\hat{x}$ convesso, quindi la soluzione $\hat{x}$ globalmente ottima.”

Note per l'esame

Domande chiave:

- “Perché TV usa L1 e non L2?”
- “Cos'è lo staircasing?”
- “Perché il problema $\hat{x}$ convesso?”

Approfondimento: Il professore potrebbe chiedere il collegamento con il corso di Ottimizzazione: la norma L1 promuove soluzioni sparse (nel gradiente), mentre la L2 promuove soluzioni dense ma piccole.

8 Slide 8 “Total Variation: Algoritmo

Slide: “Total Variation “ Algoritmo”

Collegamenti Teorici

Riferimenti: `src/methods/tv/tv.py`.

ADAM come ottimizzatore:

- Learning rate adattivo per canale: utile quando i gradienti TV hanno magnitudo variabile.
- 150 iterazioni: sufficienti per convergenza empirica.
- Clamping in $[-1,1]$ dopo ogni step: vincolo di box sul dominio.

Copione

“Vediamo l'algoritmo nel dettaglio.

Inizializziamo x con l'immagine degradata y e settiamo λ a 0.005. Usiamo l'ottimizzatore Adam con learning rate 0.001.

Ad ogni iterazione: applichiamo il blur a x , calcoliamo la loss come somma della fedeltà ai dati e della TV, facciamo backpropagation per ottenere il gradiente, aggiorniamo x con Adam, e infine clampiamo in $[-1, 1]$ per rispettare il dominio dell'immagine.

Perché Adam? Perché i gradienti della TV possono avere magnitudo variabile attraverso l'immagine $\hat{x}$ vicino ai bordi sono grandi, nelle zone uniformi sono piccoli $\hat{x}$ e Adam adatta il learning rate per canale.

Perché 150 iterazioni? Abbiamo verificato empiricamente che la loss si stabilizza entro 150 iterazioni per i nostri parametri.

Perché λ 0.005? L'abbiamo scelto come bilanciamento ottimale dopo test: a 0.001 resta troppo rumore, a 0.1 lo staircasing $\hat{x}$ evidente, a 0.005 otteniamo il miglior compromesso.”

Note per l'esame

Domanda: “Perché non usate un metodo classico come il gradient descent invece di Adam?” **Risposta:** Adam converge più velocemente e in modo più stabile grazie ai learning rate adattivi, specialmente con funzioni obiettivo non lisce come la TV (la norma L1 non è differenziabile in zero; usiamo il subgradiente).

9 Slide 9 “Total Variation: Risultati

Slide: “Total Variation “Risultati”

Collegamenti Teorici

Riferimenti: `src/methods/tv/README.md`, `report.md` §4.4.

Risultati quantitativi:

- $\sigma_n = 0.005$: PSNR 32.09 dB, SSIM 0.911 “eccellente.
- $\sigma_n = 0.1$: PSNR 26.54 dB, SSIM 0.586 “degrado significativo.
- La TV eccelle a basso rumore perché il prior TV cattura bene le caratteristiche delle immagini naturali (piecewise smooth).
- Ad alto rumore, la TV non riesce a discriminare rumore da dettagli fini.

Copione

“I risultati della TV sono chiari: a basso rumore, $\sigma = 0.005$ e 0.01 , la TV ottiene PSNR superiori a 32 dB e SSIM sopra 0.9. È un risultato eccellente per un metodo che non richiede alcun training.

Tuttavia, la qualità degrada progressivamente con il rumore. A $\sigma = 0.1$, il PSNR scende a 26.54 dB e lo SSIM a 0.586: lo staircasing diventa evidente e la TV non riesce più a distinguere il rumore dai dettagli fini delle cellule.

Nell’immagine a destra si vede qualitativamente: a $\sigma = 0.005$ il restauro è quasi perfetto, ma a $\sigma = 0.1$ l’immagine appare eccessivamente squadrata e perde dettagli.”

Note per l’esame

Domanda: “Perché TV funziona meglio proprio a basso rumore?” **Risposta:** La TV assume che le immagini siano piecewise smooth, il che è vero per le immagini naturali. A basso rumore, questa assunzione è sufficiente per separare segnale da rumore. Ad alto rumore, i dettagli fini si confondono con il rumore e vengono rimossi insieme.

10 Slide 10 “UNet: Architettura

Slide: “UNet “Architettura”

Collegamenti Teorici

Riferimenti: `report.md` §4.2, 4.3, `src/methods/unet/unet.py`.

Architettura:

- Encoder: $16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ canali.
- Decoder: simmetrico con skip connections.
- Blocco DoubleConv: Conv3x3 + GroupNorm + ReLU (2 volte).
- Input: 4 canali (RGB + noise map condizionamento).
- Output: 3 canali RGB + tanh.
- 1.9 milioni di parametri.

Perché GroupNorm invece di BatchNorm? BatchNorm si comporta male con batch size piccoli; GroupNorm è più stabile e non dipende dalla dimensione del batch.

Copione

“Passiamo all’UNet, il metodo di deep learning end-to-end.

L'architettura è un encoder-decoder con skip connections. L'encoder comprime l'immagine in rappresentazioni astratte raddoppiando i canali ad ogni livello: da 16 fino a 256. Il decoder fa il percorso inverso, ricostruendo l'immagine.

Le skip connections collegano ogni livello dell'encoder al corrispondente livello del decoder: questo permette di preservare i dettagli spaziali fini, come i nuclei cellulari, che altrimenti si perderebbero nel collo di bottiglia.

Ogni blocco è una DoubleConv, cioè due convoluzioni 3 per 3 seguite da GroupNorm e ReLU. Usiamo GroupNorm invece di BatchNorm perché è più stabile con batch size piccoli e noi usiamo batch 16.

L'input ha 4 canali: RGB più una mappa del rumore per condizionare il modello sul livello di rumore. L'output è RGB con attivazione tanh per il range $[-1, 1]$.

La rete ha circa 1.9 milioni di parametri, un modello snello e gestibile su CPU.

Note per l'esame

Domande:

- “Perché GroupNorm?”
- “Perché 4 canali in input?”
- “Perché le skip connections sono importanti?”

Approfondimento: Le skip connections risolvono il problema del *vanishing gradient* e preservano informazioni spaziali che altrimenti sarebbero perse nel bottleneck.

11 Slide 11 “UNet: Training”

Slide: “UNet Training”

Collegamenti Teorici

Riferimenti: `report.md` §4.3.3, `src/methods/unet/unet.py`.

Configurazione training:

- Loss L1 (MAE): più robusta agli outlier rispetto a MSE; preserva meglio i dettagli fini.
- Adam, $lr 10^{-4}$: learning rate conservativo per convergenza stabile.
- Batch size 16: compromesso tra stima del gradiente e memoria CPU.
- 50 epoche: sufficienti per convergenza su 673 immagini.
- Multi-noise augmentation: σ_n uniforme per batch.

Copione

“Il training usa la loss L1, che abbiamo preferito alla MSE perché è più robusta agli outlier e nel restauro di immagini, gli outlier corrispondono a dettagli fini importanti che vogliamo preservare. L'ottimizzatore è Adam con learning rate conservativo 10^{-4} .

Usiamo batch size 16 e 50 epoche, sufficienti per convergere sui 673 esempi di training.

L'aspetto chiave è la multi-noise augmentation: ad ogni batch, il livello di rumore viene scelto uniformemente tra i quattro disponibili. Questo rende il modello robusto su tutto lo spettro di rumore e previene l'overfitting su un singolo livello.

Ad ogni epoca validiamo sul validation set e salviamo il modello con il miglior PSNR. Il training è stato eseguito su CPU, circa 50 epoche in alcune ore.”

Note per l'esame

Domanda: “Perché L1 invece di MSE?” **Risposta:** La MSE è una loss L2, che penalizza quadraticamente gli errori grandi. Questo tende a produrre immagini più lisce perché il modello evita errori grandi sui bordi, sfumandoli. La L1 è più tollerante ai bordi netti e preserva meglio i dettagli.

12 Slide 12 “UNet” UNet: Risultati

Slide: “UNet” Risultati”

Collegamenti Teorici

Riferimenti: `src/methods/unet/README.md`, `report.md` §4.4.

Risultati:

- PSNR: 28.46–29.79 dB “escursione minima di circa 1 dB.
- SSIM: 0.795–0.896.
- Tempo inferenza: 0.026–0.030 s/img.
- Supera TV a $\sigma_n = 0.1$ (28.46 vs 26.54 dB).

Copione

“I risultati dell’UNet sono notevoli per due aspetti. Primo: la degradazione tra il livello di rumore più basso e quello più alto “ minima, circa 1 dB. Questo significa che UNet “ robusto su tutto lo spettro di rumore, grazie proprio alla multi-noise augmentation.

Secondo: l’inferenza “ velocissima “ circa 0.03 secondi per immagine, oltre 200 volte più veloce della TV e 100 volte più veloce di DiffPIR.

Qualitativamente, l’UNet produce immagini visivamente buone a tutti i livelli. A $\sigma = 0.1$, supera la TV con un margine di quasi 2 dB di PSNR.

Tuttavia, “ interessante notare che a basso rumore $\sigma = 0.005$ e 0.01 , l’UNet non raggiunge la TV in PSNR. Questo perché il prior appreso generalizza ma non “ specifico per ogni livello di rumore, mentre la TV con λ ottimale “ più precisa a basso rumore.”

Note per l’esame

Domanda: “Perché la TV supera UNet a basso rumore?” **Risposta:** La TV “ un modello deterministico con parametri ottimizzati per ogni σ ; l’UNet generalizza su tutti i livelli e paga un piccolo costo in precisione a basso rumore in cambio di robustezza ad alto rumore.

13 Slide 13 “DiffPIR: Panoramica

Slide: “DiffPIR “ Panoramica”

Collegamenti Teorici

Riferimenti: report.md §5.1, theory.md §3.3.

Concept:

- DiffPIR (Y. Zhu et al., CVPR 2023) integra un modello DDPM in un framework Plug-and-Play (P&P).
- P&P: separa il prior (denoising) dalla fedeltà ai dati, alternando i due step.
- Il prior nel nostro caso è un DDPM addestrato sulle nostre immagini LBC.
- La fedeltà ai dati è risolta analiticamente nel dominio di Fourier.

Copione

“Passiamo a DiffPIR, il metodo generativo implementato da Francesco.

DiffPIR è un algoritmo proposto da Zhu et al. al CVPR 2023. L’idea è integrare un modello di diffusione “ un DDPM “ dentro un framework Plug-and-Play. Plug-and-Play significa che separiamo il prior dalla fedeltà ai dati: il prior è dato dal modello di diffusione che denoisa l’immagine, mentre la fedeltà ai dati è risolta analiticamente nel dominio della frequenza.

La novità principale del nostro progetto rispetto all’articolo originale è che noi non usiamo un modello pre-addestrato su ImageNet, che ha 500 milioni di parametri e pesa 2 GB. Abbiamo invece addestrato una LightUNet custom da zero sulle nostre 673 immagini LBC. Questo modello ha solo 1.26 milioni di parametri e pesa circa 5 MB.

È specifico per il dominio citologico e giriamo sia training che inferenza su CPU.”

Note per l’esame

Domanda: “Cosa significa Plug-and-Play?” **Risposta:** In P&P, il problema inverso viene risolto con un algoritmo di splitting (es. ADMM o HQS) dove il termine di regolarizzazione viene sostituito da un denoiser esterno. Il denoiser può essere qualsiasi “ BM3D, CNN, o un DDPM “ e si alterna con un passo di data-fidelity.

14 Slide 14 “ DDPM e LightUNet

Slide: “DDPM e LightUNet”

Collegamenti Teorici

Riferimenti: theory.md §3.3, src/methods/diffpir/model.py.

DDPM (Ho et al., NeurIPS 2020):

- Forward: $q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I)$
- Reverse: $p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta)$
- Training: MSE su $\varepsilon_\theta(x_t, t)$
- Scheduler lineare: $\beta_1 = 10^{-4}$, $\beta_T = 0.02$, 1000 timestep

LightUNet:

- Canali base: $32 \rightarrow \text{encoder } 32/64/128$
- Time embedding sinusoidale (dim=128) + MLP
- GroupNorm (8 gruppi) + SiLU
- Training: 30 epoche, 673 immagini LBC

Copione

“Approfondiamo il modello di diffusione.

Il DDPM definisce un processo forward in cui aggiungiamo gradualmente rumore gaussiano all’immagine per 1000 timestep, secondo uno scheduler lineare che parte da $\beta_1 = 10^{-4}$ e arriva a $\beta_T = 0.02$. Il modello impara a invertire questo processo: dato x_t rumoroso, predice il rumore ε che è stato aggiunto.

La nostra LightUNet ha una struttura più snella della UNet usata per il restauro: canali base 32, tre livelli encoder da 32 a 64 a 128, skip connections, time embedding sinusoidale con MLP per codificare il timestep, GroupNorm e SiLU come attivazione.

L'abbiamo addestrata per 30 epoche sulle 673 immagini LBC con la loss MSE standard del DDPM: il modello riceve x_t e il timestep t , e deve predire il rumore ε che è stato aggiunto."

Note per l'esame

Domande possibili:

- "Come funziona il forward process?"
- "Cosa predice esattamente il modello?"
- "Cos'è il sinusoidal time embedding?"

Approfondimento: Il modello NON predice l'immagine x_0 ma il rumore ε . La formula per ottenere x_0 da ε : $\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t}\varepsilon)/\sqrt{\bar{\alpha}_t}$.

15 Slide 15 "Algoritmo DiffPIR"

Slide: "Algoritmo DiffPIR"

Collegamenti Teorici

Riferimenti: theory.md §3.3, report.md §5.2, src/methods/diffpir/diffpir.py.

Passi dell'algoritmo:

1. Denoising step: $\hat{x}_0^{(t)} = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$
2. Data-fidelity step (FFT):

$$\tilde{x}_0 = \mathcal{F}^{-1} \left(\frac{\mathcal{F}(y)\overline{\mathcal{F}(k)} + \rho_t \mathcal{F}(\hat{x}_0)}{|\mathcal{F}(k)|^2 + \rho_t} \right)$$

3. DDIM step: $x_{t-1} \leftarrow$ sampling da $\tilde{x}_0$

Peso dinamico: $\rho_t = \lambda \cdot \frac{\sigma^2 \cdot \bar{\alpha}_t}{1 - \bar{\alpha}_t}$

Il data-fidelity step è la soluzione del problema di Wiener nel dominio della frequenza. A t grande ($\bar{\alpha}_t$ piccolo), ρ_t è grande e il passo privilegia il prior; a t piccolo, ρ_t è piccolo e si privilegia la fedeltà ai dati.

Copione

"L'algoritmo DiffPIR alterna due passi ad ogni iterazione.

Partiamo dall'immagine degradata y . Calcoliamo una volta la trasformata di Fourier del kernel di blur, cioè la OTF, Optical Transfer Function.

Poi, per ogni timestep t da t_{start} a 1, facciamo: primo, il denoising step: il DDPM predice il rumore ε , e da questo ricaviamo una stima $\hat{x}_0$ dell'immagine pulita.

Secondo, il data-fidelity step: vogliamo che l'immagine sia coerente con l'osservazione y dato il blur. Questo è un problema di Wiener che si risolve analiticamente nel dominio di Fourier. L'equazione mostra la soluzione in forma chiusa.

Il parametro ρ_t controlla il bilanciamento tra prior e fedeltà ai dati. ρ_t dinamico: a timestep alti, quando l'immagine è molto rumorosa, ρ_t è grande e diamo più peso al prior. Scendendo, il peso si riduce e prevale la fedeltà ai dati.

Usiamo 15 step DDIM invece dei 1000 del DDPM completo: è un sampling sub-campionato che accelera notevolmente l'inferenza."

Note per l'esame

Domanda: "Perché la soluzione nel dominio di Fourier?" **Risposta:** Perché la convoluzione nel dominio spaziale diventa una moltiplicazione puntuale nel dominio di Fourier. Il problema di minimizzazione si riduce a un'equazione algebrica risolta punto per punto. È molto più efficiente che risolvere il problema nello spazio originale.

16 Slide 16 “DiffPIR: Risultati

Slide: “DiffPIR “ Parametri e Risultati”

Collegamenti Teorici

Riferimenti: `report.md` §5.3, `src/methods/diffpir/README.md`.

Risultati:

- $\sigma_n = 0.005$: PSNR 15.78 dB, SSIM 0.329 “ il π^1 basso.
- $\sigma_n = 0.1$: PSNR 25.46 dB, SSIM 0.766 “ il π^1 alto.
- Comportamento inverso rispetto agli altri metodi: PSNR cresce col rumore.

Copione

“I risultati di DiffPIR mostrano un comportamento sorprendente ma coerente con la teoria dei modelli generativi.

A basso rumore, $\sigma_n = 0.005$, DiffPIR ottiene solo 15.78 dB di PSNR: il modello generativo *inventa* dettagli che non esistono “ l’effetto allucinazione “ perch“ il suo prior generativo sovrascrive i dettagli fini gi“ presenti.

Man mano che il rumore aumenta, per“², il PSNR migliora progressivamente, fino a 25.46 dB a $\sigma_n = 0.1$. Questo perch“ ad alto rumore, l’informazione “ gi“ degradata e il prior generativo aiuta a ricostruire ci“² che manca.

Il punto chiave “: DiffPIR non compete con TV e UNet a basso rumore, ma mostra il potenziale dei metodi generativi proprio dove gli altri falliscono “ ad alto rumore. Inoltre, la qualit“ visiva pu“² essere migliore di quanto suggerisca il PSNR: le immagini appaiono pi“¹ naturali perch“ il modello genera texture realistiche.”

Note per l’esame

Domanda importante: “Perch“ il PSNR di DiffPIR cresce col rumore?” **Risposta:** Perch“ a basso rumore l’immagine “ gi“ quasi pulita e un modello generativo tende a *distruggere* i dettagli fini introducendo variazioni non presenti nell’originale (bias generativo). Ad alto rumore, invece, l’immagine “ talmente degradata che il prior generativo aiuta a ricostruire informazione persa, producendo un guadagno netto.

17 Slide 17 “Implementazione

Slide: “Implementazione – Struttura del Repository”

Collegamenti Teorici

Riferimenti: README.md, report.md §6.

Struttura modulare:

- src/data/ – dataset e preprocessing
- src/degradation/ – pipeline degradazione
- src/methods/tv/, unet/, diffpir/ – metodi
- src/eval/ – metriche (PSNR, SSIM)
- src/plots/ – visualizzazione
- scripts/ – script di esecuzione
- tests/ – 34 unit test

Copione

“La struttura del repository riflette la modularità del progetto.

Ogni metodo è un modulo indipendente in src/methods/. La pipeline di degradazione è condivisa in src/degradation/, così come le metriche in src/eval/ e la visualizzazione in src/plots/.

L'esecuzione segue una pipeline sequenziale: preprocessing, poi ogni metodo con il proprio script, infine la generazione dei grafici comparativi.

Abbiamo anche 34 test unitari che coprono degradazione, metriche e tutti e tre i metodi.

La riproducibilità è garantita dal seed 42, dalla pipeline unica di degradazione e dai test.”

Note per l'esame

Domanda: “Come avete garantito la riproducibilità?” **Risposta:** Seed fisso 42 in tutti i componenti stocastici; pipeline unica di degradazione condivisa; split del dataset salvato su file; 34 test unitari di verifica.

18 Slide 18 “Confronto PSNR/SSIM

Slide: “Confronto PSNR / SSIM”

Collegamenti Teorici

Riferimenti: report.md §7, theory.md §4.

Risultati consolidati:

- **Basso rumore** ($\sigma_n \leq 0.01$): TV domina (PSNR > 32 dB, SSIM > 0.9).
- **Medio rumore** ($\sigma_n = 0.05$): UNet leggermente superiore in SSIM (0.864 vs 0.837), TV migliore in PSNR (30.42 vs 29.44).
- **Alto rumore** ($\sigma_n = 0.1$): UNet nettamente migliore (28.46 dB, 0.795 SSIM).
- DiffPIR ha comportamento complementare: migliora col rumore.

Copione

“Questa è la slide centrale del confronto. Mettiamo tutte le metriche fianco a fianco in un'unica tabella.

La TV domina a basso rumore, sigma 0.005 e 0.01, con PSNR oltre 32 dB e SSIM superiore a 0.9. È la scelta migliore se il rumore è trascurabile.

A medio rumore, sigma 0.05, le posizioni iniziano a cambiare: la TV mantiene un PSNR più alto, ma l'UNet ha già un SSIM migliore” segno che la qualità percettiva dell'UNet è superiore anche se il PSNR è leggermente inferiore.

Ad alto rumore, sigma 0.1, l'UNet supera nettamente la TV: 28.46 contro 26.54 dB di PSNR, e 0.795 contro 0.586 di SSIM. Il distacco è significativo.

DiffPIR ha un comportamento complementare: parte male a basso rumore ma migliora progressivamente. A sigma 0.1 raggiunge 25.46 dB, pur rimanendo sotto UNet e TV.”

Note per l'esame

Domanda: “Cosa ci dice la differenza tra PSNR e SSIM?” **Risposta:** Il PSNR misura l'errore quadratico medio pixel per pixel; lo SSIM misura la similarit  strutturale. Un PSNR simile ma SSIM diverso indica che la qualit  percettiva   diversa anche se l'errore puntuale   simile. L'UNet a sigma 0.05 ha PSNR leggermente inferiore a TV ma SSIM migliore: significa che produce immagini visivamente pi  fedeli.

19 Slide 19   Grafico Comparativo

Slide: “Grafico Comparativo”

Collegamenti Teorici

Riferimenti: `src/plots/visualize.py`, `results/comparison.png`.

Copione

“Il grafico riassume visivamente quanto detto. Si vede chiaramente:

A sinistra, il PSNR: le linee di TV e UNet sono quasi piatte, mentre DiffPIR ha una pendenza crescente   l'unico metodo che migliora col rumore.

A destra, lo SSIM: TV crolla dopo sigma 0.05 perch  lo staircasing distrugge la similarit  strutturale, mentre UNet mantiene uno SSIM elevato su tutto lo spettro.

Notate l'incrocio tra TV e UNet intorno a sigma 0.05 sia in PSNR che in SSIM:   il punto in cui il metodo classico cede il passo al deep learning.”

Note per l'esame

Domanda: “Perch  le linee di TV e UNet sono quasi piatte in PSNR ma divergono in SSIM?” **Risposta:** Il PSNR   poco sensibile alla struttura dell'immagine; anche immagini con artefatti visibili possono avere PSNR decenti. Lo SSIM cattura invece la degradazione strutturale (staircasing, blur) e quindi mostra un divario maggiore tra i metodi.

20 Slide 20 “Griglia Completa”

Slide: “Griglia Completa”

Copione

“Questa slide mostra una griglia che confronta visivamente i tre metodi per diversi livelli di rumore. Ogni riga “ un metodo, ogni colonna un livello di rumore.

Si vede a colpo d’occhio: la TV “ pulita a sinistra ma degrada verso destra; l’UNet “ uniforme su tutta la griglia; DiffPIR “ rumoroso a sinistra ma migliora verso destra.

La griglia conferma visivamente quanto visto nelle metriche quantitative.”

21 Slide 21 “Confronto Visivo $\sigma_n = 0.01$ ”

Slide: “Confronto Visivo “ $\sigma_n = 0.01$ ”

Collegamenti Teorici

Riferimenti: `results/tv/qualitative/`, `results/unet/qualitative/`, `results/diffpir/qualitative/`, `src/plots/visualize.py`.

Copione

“A sigma 0.01, il rumore “ molto basso. TV e UNet producono risultati visivamente eccellenti, quasi indistinguibili dalla ground truth sia a occhio nudo che nei dettagli.

DiffPIR mostra già artefatti generativi “ l’immagine appare alterata, come se il modello stesse *ridisegnando* texture che invece nell’originale erano già presenti. “ il sintomo dell’allucinazione a basso rumore.

In basso, un crop ingrandito mette in evidenza le differenze nei dettagli fini dei nuclei cellulari: TV e UNet preservano i bordi netti, DiffPIR li sfuma o li altera.”

Note per l’esame

Nota: A $\sigma_n = 0.01$, il rumore “ appena percettibile. Questa slide serve a dimostrare che un metodo generativo **non va usato** quando il rumore “ trascurabile.

22 Slide 22 “Confronto Visivo $\sigma_n = 0.05$ ”

Slide: “Confronto Visivo “ $\sigma_n = 0.05$ ”

Collegamenti Teorici

Riferimenti: `results/tv/qualitative/`, `results/unet/qualitative/`, `results/diffpir/qualitative/`.

Copione

“A sigma 0.05, il rumore “ moderato e qui si vede il primo cambio di regime.

La TV inizia a mostrare staircasing evidente nelle zone uniformi dello sfondo “ l’immagine appare *squadrata*. I dettagli fini delle cellule sono ancora riconoscibili ma con bordi artefatti.

L’UNet mantiene una buona qualità : il rumore “ rimosso efficacemente e la struttura cellulare “ preservata. “ il punto in cui il deep learning inizia a mostrare il suo vantaggio rispetto al metodo classico.

DiffPIR migliora visibilmente rispetto a sigma 0.01: le texture generate sono più plausibili e la struttura globale “ coerente.”

Note per l’esame

Domanda: “A sigma 0.05, TV ha ancora PSNR più alto di UNet nella tabella. Come si spiega?”
Risposta: Il PSNR misura l'errore puntuale, non la qualità percettiva. La TV preserva meglio i pixel individuali ma produce artefatti strutturali (lo staircasing) che lo SSIM cattura (0.837 vs 0.864 per UNet).

23 Slide 23 “Confronto Visivo $\sigma_n = 0.1$ ”

Slide: “Confronto Visivo “ $\sigma_n = 0.1$ ”

Collegamenti Teorici

Riferimenti: results/diff/all_diffs_0.1.png (mappe di differenza).

Copione

“A sigma 0.1, il rumore è alto e le differenze tra i metodi sono massime.

La TV produce immagini squadrate con staircasing molto evidente: le zone uniformi sono segmentate in blocchi, i nuclei perdono i contorni naturali. Il PSNR scende a 26.54 dB e lo SSIM crolla a 0.586.

L'UNet resta pulito e riconoscibile nonostante l'alto rumore: è il miglior risultato sia quantitativamente (28.46 dB) che qualitativamente.

DiffPIR, pur con PSNR più basso (25.46 dB), produce texture realistiche che visivamente possono essere preferibili alla TV: l'immagine appare *naturale* anche se non identica pixel-per-pixel all'originale.

Le mappe di differenza in basso sono rivelatrici: la TV ha errori diffusi e strutturati (pattern a blocchi); l'UNet ha errori minimi e isotropi; DiffPIR ha errori concentrati dove il modello ha *allucinato* dettagli.”

Note per l'esame

Domanda: “Quale metodo produce l'immagine più naturale a sigma 0.1?” **Risposta:** Soggettivo. L'UNet è oggettivamente più fedele (PSNR/SSIM). DiffPIR può sembrare più naturale perché genera texture coerenti, ma può introdurre pattern inesistenti (allucinazioni). La TV è artificiale a causa dello staircasing.

24 Slide 24 “Confronto tra le Famiglie

Slide: “Confronto tra le Famiglie”

Collegamenti Teorici

Riferimenti: report.md §7.3, report/report.md discussione.

Copione

“Questa slide riassume pro e contro di ogni famiglia.

TV: non richiede dati, è interpretabile, robusto a basso rumore. Ma soffre di staircasing e degrada rapidamente.

UNet: inferenza velocissima, robusto su tutto lo spettro. Ma richiede training supervisionato e generalizza solo sul dominio di training.

DiffPIR: ricostruisce dettagli persi, migliora col rumore. Ma è lento e produce allucinazioni a basso rumore.

Le metriche chiave a destra mostrano il PSNR a sigma 0.005, dove TV guida.”

Note per l'esame

Domanda: “Se dovete consigliare un metodo per un'applicazione reale?” **Risposta:** Dipende dal contesto. In uno screening clinico dove il costo dell'errore è alto, UNet offre la migliore robustezza generalizzata. TV come baseline gratuita per conferma. DiffPIR solo se il rumore è notoriamente alto e serve ricostruzione generativa.

25 Slide 25 “Regimi di Funzionamento”

Slide: “Regimi di Funzionamento”

Collegamenti Teorici

Riferimenti: `report.md` §7.4 (analisi comparativa per regime).

Copione

“La tabella riassume i regimi di funzionamento.

A basso rumore, TV “eccellente”, UNet buono, DiffPIR scarso. A medio rumore, TV “buono”, UNet molto buono, DiffPIR adeguato. Ad alto rumore, TV “mediocre”, UNet “il migliore”, DiffPIR “buono”.

In termini di velocità: UNet domina con 0.035 secondi per immagine, DiffPIR impiega circa 3 secondi, TV circa 7 secondi.

Il messaggio chiave: nessun metodo domina tutti i regimi. La scelta dipende dal contesto applicativo. L’UNet “il miglior compromesso complessivo”. DiffPIR “utile solo ad alto rumore”. TV “una scelta pragmatica quando il rumore “basso e non si hanno dati.”

Note per l’esame

Domanda: “Cosa consigliate per un sistema reale?” **Risposta:** Un sistema ibrido: TV come filtro rapido per immagini a basso rumore, UNet come default, DiffPIR come opzione per casi difficili. Si potrebbe anche pensare a un meta-modello che sceglie il metodo in base al livello di rumore stimato.

26 Slide 26 “Conclusioni”

Slide: “Conclusioni”

Copione

“Concludiamo riassumendo i risultati principali.

Abbiamo confrontato equamente tre famiglie metodologiche “variazionale, deep learning, generativo” sulla stessa pipeline, con le stesse metriche e lo stesso test set. Abbiamo 34 test unitari a garanzia della correttezza.

Lezioni apprese: un problema ill-posed non significa irrisolvibile; ci sono compromessi diversi tra qualità, velocità e generalizzazione. I metodi generativi vanno usati con cautela a basso rumore perché possono allucinare dettagli. La CPU ha limitato la scala degli esperimenti “con GPU si potrebbero esplorare modelli π^1 grandi.

Direzioni future: tuning adattivo di λ per TV in funzione di σ , training GPU per modelli π^1 grandi, validazione su immagini cliniche reali per testare la generalizzazione.”

Note per l’esame

Domanda: “Cosa cambiereste se poteste ricominciare?” **Risposta:** Useremmo GPU per training π^1 lunghi e modelli π^1 grandi per UNet e DiffPIR, e introdurremmo un metodo adattivo per la scelta automatica del metodo basata sul rumore stimato.

27 Slide 27 “Riferimenti”

Slide: “Riferimenti”

Collegamenti Teorici

Bibliografia principale:

1. Y. Zhu et al., “Denoising Diffusion Models for Plug-and-Play Image Restoration”, CVPR 2023. (*DiffPIR*)

2. J. Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020. (*DDPM*)
3. J. Song et al., “Denoising Diffusion Implicit Models”, ICLR 2021. (*DDIM*)
4. L. Rudin, S. Osher, E. Fatemi, “Nonlinear total variation based noise removal algorithms”, Physica D 1992.
5. O. Ronneberger et al., “U-Net: Convolutional Networks for Biomedical Image Segmentation”, MICCAI 2015.
6. Mendeley LBC Cervical Cancer dataset, <https://data.mendeley.com/datasets/zddtpgzv63/2>.

Copione

Qui sono elencati i riferimenti principali. I più importanti per il progetto sono l’articolo di Rudin–Osher–Fatemi per la TV, Ronneberger per l’UNet, e Zhu et al. per DiffPIR. Il dataset Mendeley è disponibile al link indicato.

Grazie per l’attenzione. Siamo pronti per le domande.”